

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH**



MÔN HỌC: ĐỒ HỌA MÁY TÍNH

Cài đặt môi trường, vẽ hình hộp và mặt phẳng sử dụng thư viện Three.js

Giảng viên hướng dẫn: TS. CÁP PHẠM ĐÌNH THẮNG

Sinh viên thực hiện: Nguyễn Thế Luân

Mã sinh viên: 23520899

Lớp: CS105.Q22

Thành phố Hồ Chí Minh, tháng 04 năm 2026

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the entire width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

GVHD

TS. Cáp Phạm Đình Thăng

Mục lục

1	Giới thiệu	3
2	Mục tiêu	3
3	Cơ sở lý thuyết	4
3.1	Giới thiệu về Three.js	4
3.2	Các thành phần cơ bản trong Three.js	4
3.2.1	Scene	4
3.2.2	Camera	4
3.2.3	Renderer	4
3.2.4	Geometry	4
3.2.5	Material	4
3.2.6	Mesh	4
3.3	Một số đối tượng sử dụng trong bài lab	4
3.3.1	Hình hộp (Cube)	4
3.3.2	Mặt phẳng (Plane)	4
4	Nội dung thực hiện	5
4.1	Khởi tạo Scene	5
4.2	Tạo các đối tượng hình học	5
4.2.1	Tạo hình hộp	5
4.2.2	Tạo mặt phẳng	5
4.3	Thiết lập vị trí các đối tượng	6
4.4	Thêm đối tượng vào Scene	6
4.5	Thiết lập Camera	6
4.6	Khởi tạo Renderer và hiển thị	6
4.7	Hàm khởi tạo chương trình	7
5	Kết quả	7
6	Demo và mã nguồn	7

Danh sách hình vẽ

1	Kết quả hiển thị scene gồm hình hộp và mặt phẳng	7
---	--	---

1 Giới thiệu

Trong những năm gần đây, đồ họa 3D trên nền tảng web ngày càng phát triển mạnh mẽ, cho phép xây dựng các ứng dụng trực quan, sinh động ngay trên trình duyệt mà không cần cài đặt phần mềm bổ sung. Một trong những công nghệ quan trọng hỗ trợ điều này là WebGL, tuy nhiên việc sử dụng WebGL thuần khá phức tạp và đòi hỏi kiến thức chuyên sâu.

Để đơn giản hóa quá trình phát triển, thư viện Three.js đã được xây dựng như một lớp trung gian, giúp lập trình viên dễ dàng tạo và quản lý các đối tượng 3D thông qua các thành phần cơ bản như Scene, Camera, Renderer và Mesh. Nhờ đó, việc xây dựng một không gian 3D trở nên trực quan và dễ tiếp cận hơn.

Trong bài Lab-04 này, mục tiêu chính là làm quen với cách cài đặt môi trường lập trình sử dụng Three.js và xây dựng một scene 3D đơn giản. Cụ thể, chương trình sẽ tạo một hình hộp (cube) và một mặt phẳng (plane), đồng thời hiển thị và thực hiện animation cơ bản trên các đối tượng này.

Thông qua bài thực hành, sinh viên có thể nắm được quy trình cơ bản để xây dựng một ứng dụng đồ họa 3D trên web, từ đó làm nền tảng cho các bài toán phức tạp hơn trong tương lai.

2 Mục tiêu

Bài Lab-04 được thực hiện nhằm đạt được các mục tiêu sau:

- Làm quen với thư viện **Three.js** trong việc xây dựng đồ họa 3D trên nền web.
- Hiểu và sử dụng các thành phần cơ bản của một scene 3D như:
 - Scene
 - Camera
 - Renderer
 - Mesh
- Cài đặt và thiết lập môi trường lập trình sử dụng Three.js.
- Xây dựng một scene 3D đơn giản bao gồm:
 - Một hình hộp (cube)
 - Một mặt phẳng (plane)
- Thực hiện hiển thị cơ bản cho đối tượng trong không gian 3D.
- Làm nền tảng cho việc phát triển các ứng dụng đồ họa 3D phức tạp hơn trong các bài lab tiếp theo.

3 Cơ sở lý thuyết

3.1 Giới thiệu về Three.js

Three.js là một thư viện JavaScript mã nguồn mở hỗ trợ xây dựng đồ họa 3D trên nền tảng web thông qua WebGL. Thư viện này cung cấp các công cụ giúp đơn giản hóa việc tạo, hiển thị và quản lý các đối tượng 3D trong trình duyệt.

3.2 Các thành phần cơ bản trong Three.js

3.2.1 Scene

Scene là nơi chứa toàn bộ các đối tượng trong không gian 3D, bao gồm các mô hình, ánh sáng và camera. Mọi đối tượng muốn hiển thị đều phải được thêm vào Scene.

3.2.2 Camera

Camera xác định góc nhìn của người quan sát trong không gian 3D. Trong Three.js, loại camera phổ biến là PerspectiveCamera, mô phỏng cách nhìn của mắt người với hiệu ứng phối cảnh.

3.2.3 Renderer

Renderer có nhiệm vụ hiển thị Scene thông qua Camera lên màn hình. Trong Three.js, WebGLRenderer được sử dụng để tận dụng khả năng xử lý đồ họa của GPU.

3.2.4 Geometry

Geometry định nghĩa hình dạng của đối tượng 3D, ví dụ như hình hộp (BoxGeometry), mặt phẳng (PlaneGeometry), hình cầu,...

3.2.5 Material

Material xác định bề mặt của đối tượng như màu sắc, độ bóng hoặc kết cấu. Ví dụ: MeshBasicMaterial cho phép tô màu đơn giản cho đối tượng.

3.2.6 Mesh

Mesh là sự kết hợp giữa Geometry và Material để tạo thành một đối tượng 3D hoàn chỉnh có thể hiển thị trong Scene.

3.3 Một số đối tượng sử dụng trong bài lab

3.3.1 Hình hộp (Cube)

Hình hộp được tạo bằng BoxGeometry với ba tham số là chiều dài, chiều rộng và chiều cao.

3.3.2 Mặt phẳng (Plane)

Mặt phẳng được tạo bằng PlaneGeometry, thường được sử dụng làm nền trong không gian 3D.

4 Nội dung thực hiện

4.1 Khởi tạo Scene

Đầu tiên, tạo một Scene để chứa các đối tượng 3D.

```
const scene = new THREE.Scene();
```

Giải thích: Scene là không gian chính để thêm và quản lý các đối tượng trong môi trường 3D.

4.2 Tạo các đối tượng hình học

Trong bài lab, hai đối tượng chính được sử dụng là hình hộp (cube) và mặt phẳng (plane). Các đối tượng này được tạo thông qua hai hàm riêng biệt.

4.2.1 Tạo hình hộp

```
function getBox(w, h, d) {  
    var geometry = new THREE.BoxGeometry(w, h, d);  
    var material = new THREE.MeshBasicMaterial({ color: 0x00ff00 });  
    var mesh = new THREE.Mesh(geometry, material);  
    return mesh;  
}
```

Giải thích:

- BoxGeometry tạo hình hộp với kích thước (w, h, d).
- MeshBasicMaterial dùng để tô màu xanh cho hình hộp.
- Mesh là đối tượng hoàn chỉnh được tạo từ geometry và material.

4.2.2 Tạo mặt phẳng

```
function getPlane(size) {  
    var geometry = new THREE.PlaneGeometry(size, size);  
    var material = new THREE.MeshBasicMaterial({  
        color: 0xff0000,  
        side: THREE.DoubleSide  
    });  
    var mesh = new THREE.Mesh(geometry, material);  
    return mesh;  
}
```

Giải thích:

- PlaneGeometry tạo mặt phẳng với kích thước cho trước.
- Thuộc tính DoubleSide giúp hiển thị cả hai mặt của mặt phẳng.

4.3 Thiết lập vị trí các đối tượng

```
mesh.position.y = mesh.geometry.parameters.height / 2;  
plane.rotation.x = Math.PI / 2;
```

Giải thích:

- Đưa hình hộp lên trên để không bị chìm xuống mặt phẳng.
- Xoay mặt phẳng 90 độ để nằm ngang làm nền.

4.4 Thêm đối tượng vào Scene

```
scene.add(plane);  
scene.add(mesh);
```

Giải thích: Các đối tượng phải được thêm vào Scene thì mới có thể hiển thị.

4.5 Thiết lập Camera

```
var camera = new THREE.PerspectiveCamera(  
    45, window.innerWidth / window.innerHeight, 0.1, 1000  
);  
  
camera.position.x = 1;  
camera.position.y = 2;  
camera.position.z = 5;  
  
camera.lookAt(new THREE.Vector3(0, 0, 0));
```

Giải thích:

- Camera được đặt ở vị trí lệch để quan sát rõ đối tượng.
- Hàm lookAt giúp camera hướng về tâm của scene.

4.6 Khởi tạo Renderer và hiển thị

```
var renderer = new THREE.WebGLRenderer();  
renderer.setSize(window.innerWidth, window.innerHeight);  
document.getElementById("webgl").appendChild(renderer.domElement);  
  
renderer.render(scene, camera);
```

Giải thích:

- Renderer dùng để hiển thị scene lên trình duyệt.
- Scene được render một lần (không có animation).

4.7 Hàm khởi tạo chương trình

```
function init() {  
    ...  
}  
init();
```

Giải thích: Hàm `init()` dùng để tổ chức toàn bộ chương trình và được gọi để bắt đầu chạy ứng dụng.

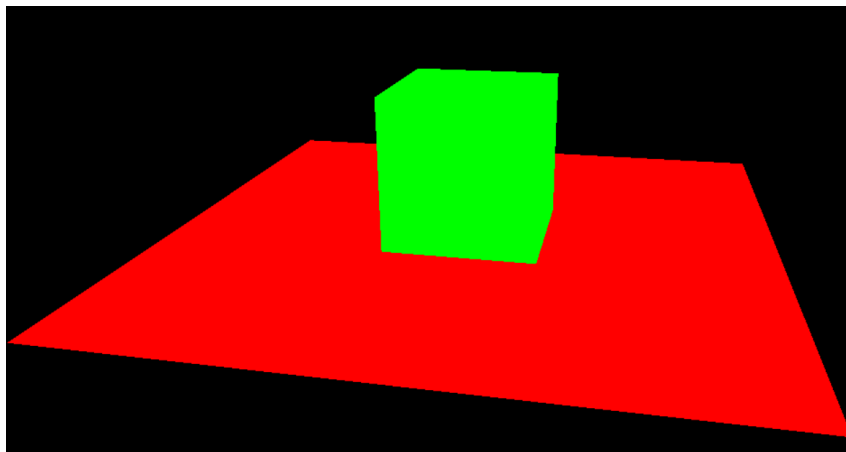
5 Kết quả

Sau khi thực hiện các bước cài đặt và xây dựng chương trình, kết quả thu được là một scene 3D đơn giản hiển thị trên trình duyệt.

Cụ thể:

- Một hình hộp (cube) màu xanh được đặt trên mặt phẳng.
- Một mặt phẳng màu đỏ đóng vai trò làm nền.
- Camera được đặt ở vị trí phù hợp để quan sát toàn bộ scene.

Hình ảnh kết quả chương trình được thể hiện như sau:



Hình 1: Kết quả hiển thị scene gồm hình hộp và mặt phẳng

Nhận thấy rằng chương trình đã hiển thị đúng các đối tượng với vị trí và góc nhìn như mong muốn. Hình hộp được đặt nằm trên mặt phẳng, không bị chồng lấp hay sai lệch vị trí.

6 Demo và mã nguồn

Để minh họa cho kết quả của bài lab, chương trình đã được triển khai và chia sẻ thông qua các đường dẫn sau:

- **Link web demo:** <https://cs-105-q22.vercel.app/>
- **Link GitHub:** CS105.Q22 - Lab-04
- **Video demo:** Lab 04 - Video demo